



Alunos: Gabriel Antonio Gomes Moutinho
Alunos: Gabriel Henrique Carneiro Amorim

Desafio 1

Inteligência Computacional



Desafio



Mapa de características para o Perceptron
GBC073 — Inteligência Computacional (FACOM/UFU)

O QUE VOCÊ FAZ: preencher a classe `Submissao` (e só ela).

O QUE O HARNESS FAZ: gera dados, divide 60/40, aplica a sua ϕ , treina um Perceptron fixo e mede a acurácia. Rode: `python desafio1_aluno.py`

Regras:

- * `fit(X)` recebe só as entradas de treino, sem rótulos. É opcional.

- * `$\phi(X)$` transforma (n, d) em (n, d') com $d < d' \leq 64$, de forma determinística.

- * Sem NaN/Inf; e rápido (10 mil pontos em menos de 2 s).

Score: 0 = igual à identidade (baseline), 100 = igual à referência do professor,

até 125 se superar a referência. Na correção, tarefas OCULTAS da mesma família

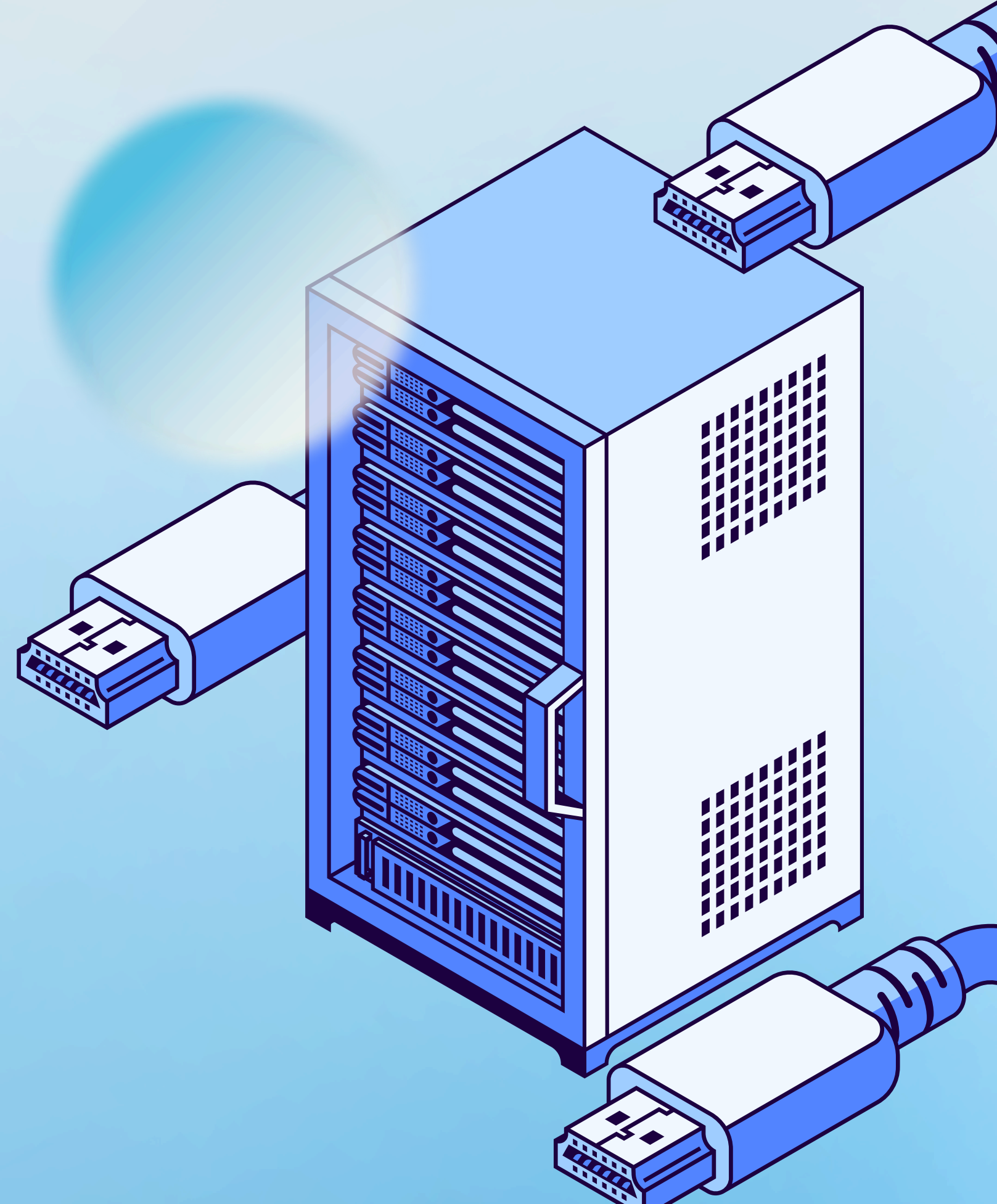
substituem estas — não ajuste para um conjunto de dados específico.

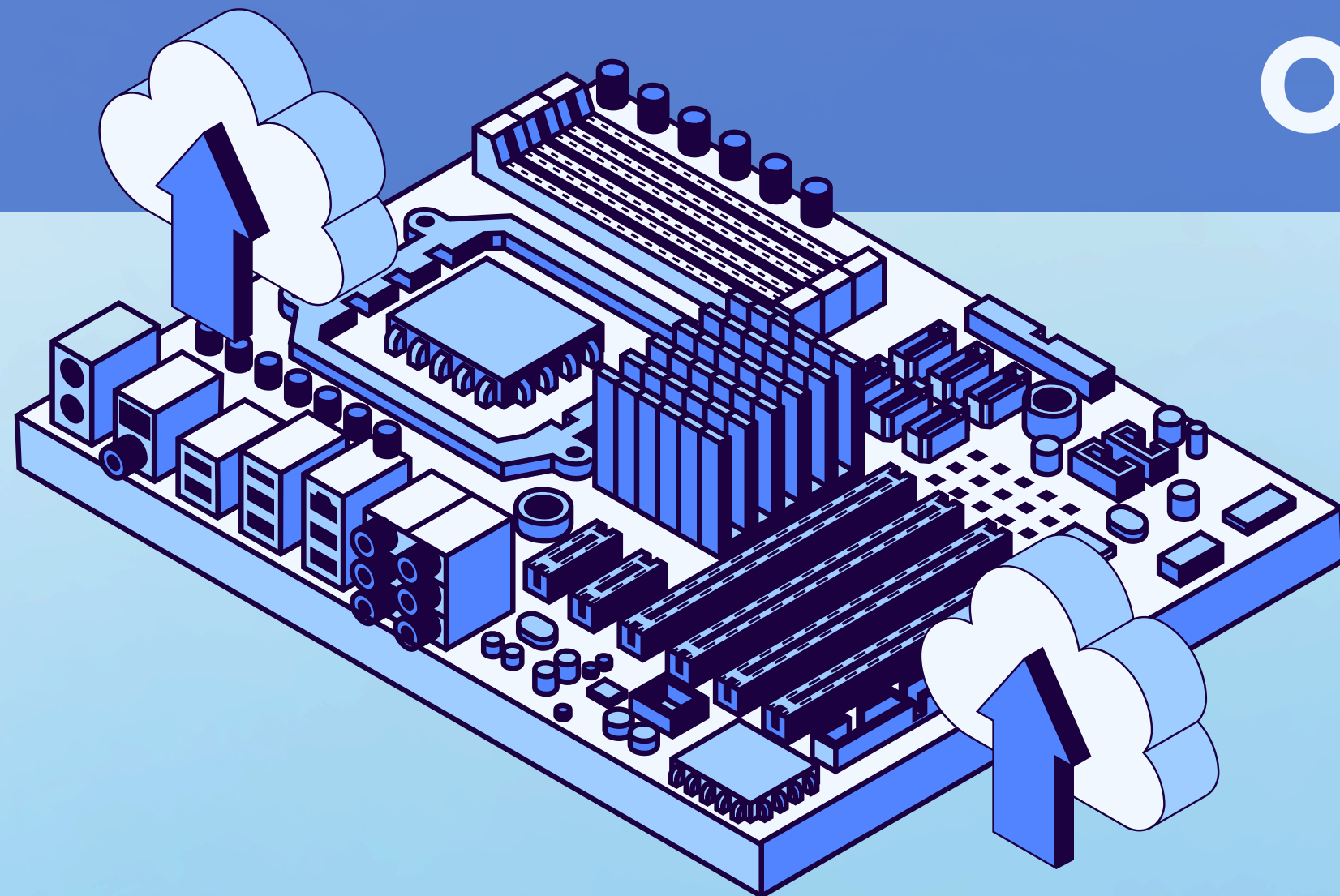
O que tentamos?

De início, para chegar à quantidades maiores de dimensões, resolvemos usar matrizes de pesos aleatórios e a função de ativação. Definimos os pesos aleatórios com `torch.randn` e fixamos a seed com um valor fixo.

Aplicamos a função de ativação `torch.relu` e ao testarmos o código a primeira vez deu tudo certo e o score S até apresentou um resultado promissor. Porém, após um segundo teste, a saída de algumas das formas começou a apresentar como “phi não é determinística” e nada que fizemos corrigiu inicialmente.

Trocamos a função de ativação para `torch.tanh` porém o resultado caiu ainda mais.



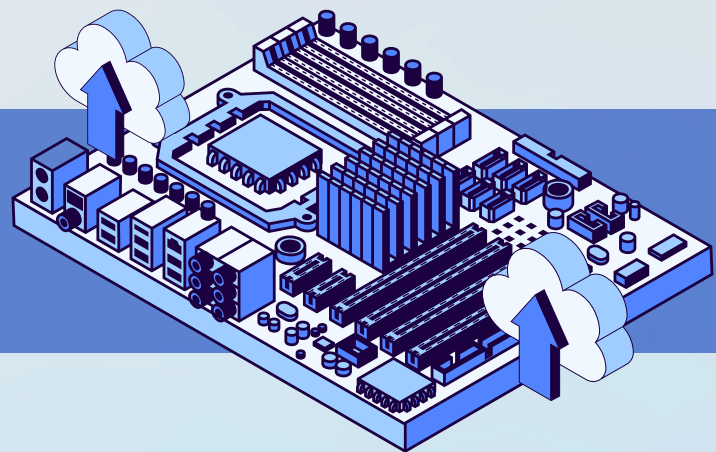


O que fizemos?

Imaginamos que, devido à saída relacionada à “phi não é determinístico” de algumas formas, que o problema estava relacionada ao processamento do ponto flutuante.

Tentamos por algumas vezes realizar a multiplicar por valores muito pequenos (0.01) para tentar forçar a adequação e realmente resolveu esse problema porém nosso score ainda continuava ruim.

Então, resolvemos mudar a abordagem e dividimos o problema.



O que fizemos?

Realizamos separadamente o cálculo dos quadrados dos dados originais ($Xs ** 2$), resolvendo as formas relacionadas aos Círculos e Esfera, sem considerar os pesos aleatórios:

```
quadrados = Xs ** 2
```

Usamos as dimensões restantes para criar uma camada oculta com seed fixa e usamos a função de ativação ReLU, com isso resolvemos o restante das formas (Espiral, Luas e XOR).

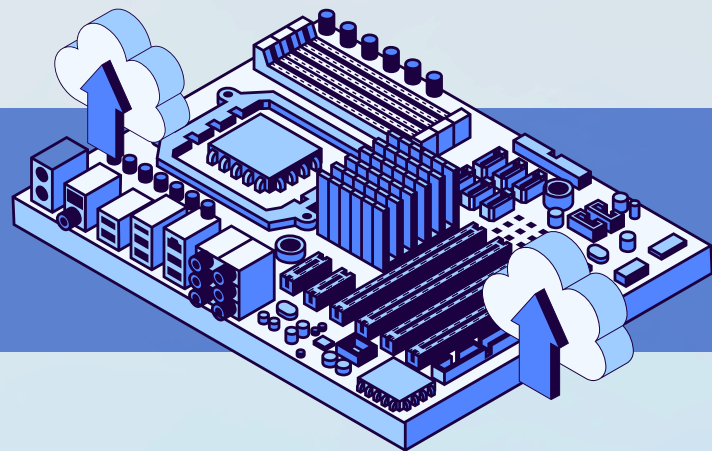
```
self.k = self.DIM_MAX - (2 * d)

# usando uma seed fixa para garantir o determinismo
g = torch.Generator().manual_seed(102)
self.W = torch.randn(d, self.k, generator=g)
self.b = torch.randn(self.k, generator=g)
```

```
# mapeamento geométrico para camada oculta simulada com ReLU que
# resolve as formas espiral, luas e XOR através da dobra e segmentação do espaço
ativacao_relu = torch.relu(Xs @ self.W + self.b)
```

Pra finalizar, concatenamos os dados, os quadrados e a saída da função ReLU em um único tensor, gerando um Escore S superior à referência.

```
return torch.cat([Xs, quadrados, ativacao_relu], dim=1)
```



Resultado

```
cpu = _conversion_method_template(device=torch.device("cpu"))
tarefa      baseline  referência  você      s_t
xor          0.606     0.876     0.897     1.08
duas_luas    0.892     0.983     0.983     1.00
circulos     0.644     1.000     0.996     0.99
espiral      0.633     0.912     0.916     1.01
esfera_10d   0.508     0.872     0.914     1.11

ESCORE S = 102.3 (0 = baseline, 100 = referência, até 125 com bônus)
```

Apresentou um resultado de 102.3 (superior ao de referência)